



EAST WEST UNIVERSITY

Mini Project Report

Dynamic Spatial–Temporal Graph Transformer for EEG-Based Valence–Arousal Emotion Recognition on the DEAP Dataset

Course: ICE478

Course Title: Machine Learning

Section: 06

Group: 09

Course Instructor: Dr. Raihan Ul Islam

Team Members:

Name	ID
Tahsina Tabassum	2023-1-50-002
Saima Akter Tisha	2022-3-50-018
Fariya Ahmed Bhuiyan	2022-3-50-009
Iftekhar Alam	2019-2-57-039

Proposed Model Report

GCN-Based Model for EEG Valence Classification (DEAP Dataset)

1. What Changed From the Baseline

This notebook keeps the exact same data pipeline as the baseline — same band-power features, same subject-grouped split, same 32-node electrode graph — and swaps out the model itself. The baseline used a hand-written message-passing layer with separate self and neighbor weights. This version replaces it with two standard GCNConv layers from PyTorch Geometric, adds batch normalization after each convolution, widens the hidden dimension from 32 to 64, and replaces the single linear output layer with a small two-layer MLP head ($64 \rightarrow 32 \rightarrow 2$). The intent behind these changes is the kind of thing you'd expect from a model labeled 'proposed': a more standard, better-regularized architecture that should, in principle, have an easier time learning than the from-scratch baseline layer.

2. Data and Graph Recap

Nothing changes here from the earlier notebooks, so this is a brief pointer rather than a repeat. EEG from 19 of the 32 DEAP subjects was split into 4-second windows and reduced to 32×5 band-power features per window (theta through gamma, log-transformed), giving 11,400 windows in total. The subject-grouped split sends 15 subjects (9,000 windows) to training and 4 subjects (2,400 windows) to testing, with no subject appearing in both. The same $k = 5$ nearest-neighbor electrode graph (32 nodes, 160 edges, Gaussian distance weights) is attached to every window.

Item	Value
Feature shape per window	32 nodes $\times$ 5 band-power features
Train / test windows	9,000 / 2,400
Train / test subjects	15 / 4 (disjoint)
Valence balance (train+test)	8,850 low / 2,550 high ($\approx 22\%$ high)
Graph structure	32 nodes, 160 edges ($k = 5$)

3. Model Architecture

EEG_GCN stacks two GCNConv layers ($5 \rightarrow 64 \rightarrow 64$), each followed by batch normalization and ReLU, with dropout ($p = 0.3$) applied after the first block. Node embeddings are pooled into a single graph-level vector with global mean pooling, then passed through a small MLP head — a 64-to-32 linear layer, ReLU, dropout, and a final 32-to-2 linear layer for the class logits. Compared to the baseline's single linear head, this gives the classifier itself an extra layer of nonlinearity to work with.

Component	Baseline	Proposed
Graph layer	Custom MessagePassing (self + neighbor linear)	GCNConv $\times 2$ (PyTorch Geometric)
Hidden dimension	32	64

Normalization	None	BatchNorm1d after each conv
Output head	Single linear layer	Linear → ReLU → Dropout → Linear
Dropout	0.3	0.3
Pooling	Global mean pool	Global mean pool

4. Training Setup

Training is identical to the baseline in every hyperparameter that matters: Adam at $\text{lr} = 1\text{e-}3$ with weight decay $5\text{e-}4$, cross-entropy loss, 40 epochs, batch size 32 for training and 64 for testing. The only functional difference in the training loop is that this notebook logs metrics into a history dictionary for later plotting, which is a bookkeeping change, not a modeling one.

Epoch	Train loss	Train acc	Test loss	Test acc
5	0.528	0.773	0.548	0.787
10	0.528	0.773	0.543	0.787
15	0.525	0.773	0.547	0.787
20	0.525	0.773	0.535	0.787
25	0.525	0.773	0.548	0.787
30	0.525	0.773	0.538	0.787
35	0.523	0.773	0.580	0.787
40	0.523	0.773	0.571	0.787

Train accuracy is pinned at 0.773 for all 40 epochs and test accuracy at 0.787. Test loss actually drifts upward after epoch 20 while accuracy stays flat, which usually means the model is growing more confident in the same majority-class guess rather than learning anything new — a mild form of overfitting to a shortcut rather than to the data.

5. Evaluation Results

Final test performance: 78.8% accuracy, 0.443 macro-F1. Those are the same two numbers, to three decimal places, as the baseline model.

Class	Precision	Recall	F1-score	Support
Low	0.79	1.00	0.88	1,890
High	1.00	0.00	0.00	510

Confusion matrix: `[[1890, 0], [509, 1]]`

This is not a rounding coincidence — it's the identical confusion matrix as the baseline, down to the single high-valence window it happens to catch. The GCN, despite more parameters, batch normalization, and a deeper head, converged to exactly the same majority-class shortcut.

6. Comparison to Baseline

Metric	Baseline (custom GNN)	Proposed (GCN)
Accuracy	0.788	0.788
Macro-F1	0.443	0.443
High-valence recall	0.00 (1 / 510)	0.00 (1 / 510)
Low-valence recall	1.00 (1890 / 1890)	1.00 (1890 / 1890)

Architecturally, this is a real upgrade over the baseline — GCNConv is a better-tested layer than the hand-written version, batch normalization typically helps optimization, and a deeper head gives the classifier more capacity. None of that mattered here, because the failure isn't coming from the architecture. It's coming from the loss function having no reason to care about the minority class. A more expressive model trained the same way just finds the same shortcut a little more efficiently.

7. A Bug Worth Flagging

The dataset class attaches the electrode graph's edge weights under `edge_attr` (`Data(x=..., edge_index=..., edge_attr=edge_weight_t, y=...)`), and training correctly reads `batch.edge_attr`. The evaluation cell, however, reads `batch.edge_weight` instead — an attribute that was never set on these `Data` objects. PyTorch Geometric returns `None` for an unset attribute rather than raising an error, so `GCNConv` silently falls back to unweighted edges during evaluation only. Training therefore uses the Gaussian-weighted electrode graph, and testing effectively uses an unweighted version of the same graph.

Given the model's current behavior — predicting one class almost regardless of input — this mismatch had no visible effect on the reported metrics. It's still worth fixing before this becomes a real comparison point, since it means training and evaluation aren't technically using the same graph.

8. Why the Architecture Change Didn't Help

The class imbalance identified back in the EDA stage — about 22% high-valence windows against 78% low-valence — is still untouched by anything in this notebook. Cross-entropy loss with no class weighting rewards a model for predicting the majority class regardless of how many layers or parameters it has, and a more powerful model doesn't need more capacity to find that shortcut, just less patience. That's consistent with what happened here: train accuracy locked onto 0.773 by epoch 5 and stayed there for the rest of training.

This points to the same conclusion as the baseline report: the bottleneck in this pipeline is the loss function and label handling, not the graph layer choice. Swapping `GCNConv` for `GATConv`, adding more layers, or increasing hidden size further would likely reproduce the same result until the imbalance itself is addressed.

9. Limitations and Next Steps

- Class imbalance is still unaddressed. This should come before any further architecture changes — weighted cross-entropy, focal loss, or minority oversampling are all reasonable starting points.

- The `edge_attr` / `edge_weight` mismatch in the evaluation cell should be fixed so training and testing use the same weighted graph.
- Still only 19 of 32 subjects and valence only; arousal and the remaining 13 subjects haven't been touched by either model.
- No hyperparameter tuning was done — hidden size, learning rate, and dropout were carried over from the baseline rather than tuned for the new architecture.
- Because both models collapse to the same shortcut, this notebook can't yet say whether GCNConv actually helps. A meaningful comparison needs to happen after the imbalance is fixed.

10. Summary

The proposed model changes the graph layer, adds normalization, and deepens the classifier head, but produces the exact same accuracy, macro-F1, and confusion matrix as the baseline. That's a useful negative result: it shows the earlier collapse wasn't a quirk of the hand-written message-passing layer, and rules out architecture as the explanation. The next version of this model needs to fix the class imbalance before an architecture comparison will mean anything.